Distributed Processing and Deep Learning for One-to-Three-Day Discharge Forecasting of the Huong River at Kim Long

Dang Nguyen Giap Ho Anh Duc Mai Thi Le Huyen

Department of Information Technology, FPT University
DSP391m — Data Science Project — Fall 2026
Hue, Vietnam

***Abstract*—**The lecturer's review of this project asked for distributed data processing, model training with Apache Spark and, preferably, deep learning. This report moves the pipeline of a daily discharge forecast for the Huong River at Kim Long, Vietnam, onto a Spark standalone cluster (one master, 2 workers, Spark 4.1.3 on Docker) and evaluates neural forecasters under the protocol used for the gradient-boosting model. The cluster turns 6,544,896 hourly ERA5 rows into daily rainfall indices with the same result as a single-machine Polars pipeline, and the 54 hourly rainfall features agree bit for bit, also on the full 1998–2026 satellite record (101.0 million rows). MLlib gradient-boosted trees and random forests trained on the cluster with the main model's full data are worse than LightGBM at every horizon over 13 walk-forward folds (one day ahead –0.044 and –0.049 NSE, $p < 0.05$). An LSTM trained with PyTorch DDP through TorchDistributor or a Spark barrier stage gives the same result on one machine and on two workers. Across 13 yearly walk-forward folds no LSTM or MLP beats LightGBM consistently; a 50/50 blend of LSTM and the main LightGBM model improves NSE one day ahead by +0.029 ($p = 0.048$) but not at two or three days. At this data size the two-worker ETL takes 11.6 times as long as Polars on one machine and distributed training is slower than one process; we measure why. Where Spark does pay off is task parallelism: 141 walk-forward LightGBM runs finish 7.7 times faster than in sequence on 8 cores, with the same forecasts as local runs.

***Keywords*—**Apache Spark, MLlib, distributed deep learning, LSTM, TorchDistributor, scalability, flood forecasting, Huong River

I. INTRODUCTION

The project forecasts the daily discharge of the Huong River at Kim Long (Hue, Vietnam) one, two and three days ahead from open data, using the GloFAS v4 simulation as the target [1], [2]. Report 3 selected LightGBM [10] as the main model (NSE 0.747, 0.499 and 0.236 on the test period from July 2022) and introduced a yearly walk-forward evaluation. The lecturer's review asked that Report 4 treat the data as distributed, train the models with Apache Spark [3] and, better still, use deep learning. This report answers that request and reports what it costs.

The raw inputs are hourly (Table I). ERA5 rainfall from Open-Meteo has 6,544,896 hourly rows for 64 cells since 2015, and satellite rainfall from IMERG V07, GSMaP v8 and the near-real-time GSMaP_NRT on 100 cells since 1998 adds 101.0 million rows. In bytes the data are small: 75 MB and 125 MB of compressed Parquet, and the modelling table has one row per day. The useful questions are therefore whether the pipeline stays correct when it is split across executors, how a distributed system compares with a good single-machine implementation [4], and which part of the work actually gains from a cluster.

Contributions: (i) a Spark standalone cluster and Spark jobs that reproduce the single-machine pipeline exactly, the hourly features bit for bit on the full satellite record; (ii) MLlib gradient-boosted trees and random forests [5] trained on the cluster with a leakage-free hyperparameter search, also on the full data of the main model and scored over yearly walk-forward folds; (iii) LSTM training with PyTorch distributed data parallel (DDP) [6] through Spark, with TorchDistributor [7] and a barrier-mode fallback; (iv) a walk-forward comparison of deep learning with LightGBM, alone and as an ensemble member; (v) a task-parallel walk-forward on Spark, the one place where the cluster shortened the work; and (vi) measured run times with an explanation of why Spark is slower elsewhere.

TABLE I
HOURLY INPUT DATA (ROWS = CELL × TIME STEP; SIZE = PARQUET ON DISK)

Source	Cells (grid)	Step	Period	Rows	Size
ERA5, Open-Meteo	64 (0.1°)	1 h	2015–Aug. 2026	6,544,896	75 MB
IMERG V07, Earth Engine	100 (0.1°)	30 min	1998–Oct. 2026	50,477,000	99 MB (with GSMaP)
GSMaP v8, raw and gauge-calibrated	100 (0.1°)	1 h	1998–Oct. 2026	25,241,800	
GSMaP_NRT, JAXA	100 (0.1°)	1 h	1998–Oct. 2026	25,245,600	26.3 MB
PERSIANN-CCS (test months)	625 (0.04°)	1 h	Oct.–Nov. 2020	915,000	—
ERA5 after the Spark ETL	64 cells	1 day	2015–Aug. 2026	272,704	—

II. DISTRIBUTED ARCHITECTURE

A. Cluster

The cluster runs on Docker Desktop: one master and 2 workers with 4 cores and 2 GB each (Fig. 1), built from the official Spark 4.1.3 image with Java 21, Polars 1.17.1 and CPU PyTorch 2.14.1; the host has pyspark 4.1.3, matching the cluster. The repository is mounted at `/opt/repo` in all three

containers. This shared folder plays the role that HDFS, S3 or a Delta Lake table [8] plays in a production cluster: an executor on any worker reads and writes the same paths. All containers run on one physical machine, so the set-up is distributed in protocol (separate executors, barrier stages, gradient exchange over the Docker network) but adds no hardware.

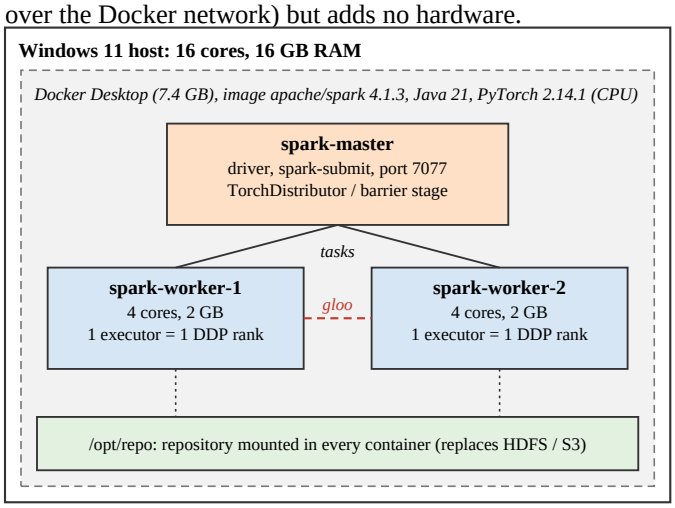


Fig. 1. Spark standalone cluster used in this report. Three containers share one physical machine; the mounted repository replaces a distributed file system.

On the Windows host itself Spark 4 cannot read local Parquet files without the Hadoop native binaries (winutils), which the team chose not to install from unofficial sources. On the host, data therefore travel through Arrow (Polars reads, Spark computes, Polars writes); this path is used only for unit tests and for one benchmark row.

B. Hourly-to-daily ETL

`spark_etl.py` reads 768 Parquet files (6,544,896 rows) and writes 272,704 cell-days (64 cells × 4,261 days), partitioned by year: daily total, maximum 1-, 3- and 6-h rain, wet hours and peak hour. Sliding windows are defined on time (`rangeBetween` in seconds), not on row counts, so a missing hour cannot stretch a window, and windows reach back into the previous day so that a storm crossing midnight is not cut in two. Sub-basin rainfall is averaged at the hourly level before the daily indices are computed (21,305 sub-basin-days). Against the Polars implementation every row matches, rain and count columns agree exactly, and mean temperature differs by 0.001°C because the two engines round halves differently. Verification found one real bug: the sub-basin peak hour disagreed between host and cluster on 79 of 21,305 rows, because weighted means differ in the last bits with the order of summation and turned ties into non-ties. Rainfall is now rounded to 0.001 mm before the peak is located, and both platforms agree on all rows. Run times are shown in Fig. 2 and discussed in Section V.

C. Hourly features: bit-identical results

`spark_hourly.py` re-implements the 54 hourly rainfall features of Report 3 (nine per sub-basin and for the whole basin). Compared cell by cell with the Polars version using exact equality, no cell differs (Table II). Exactness needed three changes, because a difference of one unit in the last place can flip a threshold test: sums are accumulated sequentially in the Polars order (`aggregate` over a sorted array instead of `F.sum`); the five sub-basins are added with the same bracketing as Polars; and the sine and cosine of the peak hour come from a table computed in Polars rather than from the JVM. Between Windows and Linux the C-library sine itself differs in the last place, in both engines; that difference is not caused by Spark.

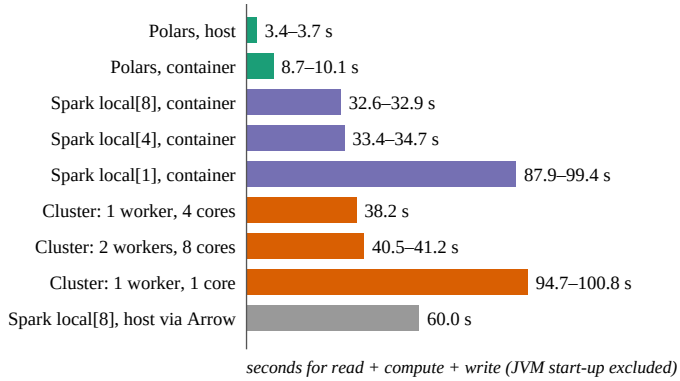


Fig. 2. Run time of the hourly-to-daily ETL (6,544,896 rows). Bars: mean of the runs; labels: minimum and maximum. Green: Polars; purple: Spark local mode; orange: standalone cluster; grey: Spark on the host via Arrow.

TABLE II
SPARK AGAINST POLARS FOR THE 54 HOURLY FEATURES: EXACT (BITWISE) COMPARISON, NO ROW ON ONE SIDE ONLY

Source, where Spark ran	Cells compared (hourly / daily)	Differing cells	Spark (s)	Polars (s)
ERA5, cluster (2 workers)	613,584 / 230,064	0	28.6	3.9
ERA5, host local[8]	613,584 / 230,064	0	42.3	1.3
PERSIANN-CCS, cluster	8,784 / 3,234	0	14.0	0.2
PERSIANN-CCS, host local[8]	8,784 / 3,234	0	35.7	0.2

D. Full satellite record on the cluster

The same job was then run on the whole satellite archive, January 1998 to October 2026: IMERG at 30 minutes (50,477,000 rows in 346 files), GSMaP-GC (25,241,800) and GSMaP_NRT (25,245,600, 364 files), in total 101.0 million rows and 125 MB. The job keeps the 24 cells that carry sub-basin weights, sums IMERG half-hours into hours and produces 252,456–252,480 hours and 10,519–10,520 days of the 54 features per product. For each product about 1.5 million hourly and 0.57 million daily values were compared with Polars, on the two-worker cluster and on the Windows host: none differs in any bit (6,246,870 values per platform), and no change beyond the three used for ERA5 was needed. The workers kept their 2 GB and the executors their default memory. The only failure was in local mode: run through `spark-submit`, the driver JVM starts before the session sets its memory, keeps a 1 GB heap shared by all eight tasks and ran out of memory on IMERG; passing `--driver-memory`

`4g` to `spark-submit` fixed it (6 of 6 runs).

TABLE III
HOURLY FEATURES ON THE FULL SATELLITE RECORD, 1998–2026: READ + COMPUTE + COLLECT, JVM START-UP EXCLUDED; MEDIAN OF 1–3 RUNS

Seconds (median)	IMERG	GSMaP-GC	GSMaP_NRT
Cluster, 2 workers × 4 cores	42.6	31.6	30.9
Cluster, 1 worker × 4 cores	40.3	31.3	29.4
Spark local[8], container	29.8	24.1	23.3
Polars, container	16.3	5.1	4.7
Polars, host	10.4	2.4	2.3
Spark local[8], host via Arrow	114.5	64.3	65.0
Differing values, cluster / host	0 / 0	0 / 0	0 / 0

Table III repeats the ERA5 picture at 15 times the rows. Two workers are no faster than one, and Polars in the same container is 1.8–6.6 times faster than Spark. IMERG is where Spark comes closest (cluster 2.6 times Polars), because summing half-hours into hours spreads evenly over tasks; after the cell filter each product is only a few million rows, and the sliding windows are partitioned by 6 spatial units, so later stages have little parallelism. The first cluster run of each product (cold cache on the mount, with the comparison) took up to 71.0 s.

III. DISTRIBUTED MACHINE LEARNING WITH MLLIB

A. *Report 2 panel, one split*

`spark_train.py` trains MLlib gradient-boosted trees (GBT) and random forests (RF) on the 66 predictors of the Report 2 panel (configuration A of Report 3), on log(1 + Q), one model per horizon, with the same temporal split as LightGBM: training 2010–2015 (2,177 days), validation to June 2022 (2,373 days), refit on both (4,550 days) and a single evaluation on the test period from July 2022 (1,520–1,522 days). Spark's `CrossValidator` and `TrainValidationSplit` split rows at random, which lets a model see the future; they were replaced by a loop that fits each grid point on the training years and scores it on the fixed validation period (8 GBT and 4 RF settings, 36 fits over three horizons).

TABLE IV
MLLIB ON THE TWO-WORKER CLUSTER AGAINST LIGHTGBM, SAME PANEL AND SPLIT, TEST PERIOD FROM JULY 2022

Model	NSE h=1	h=2	h=3	Peak bias h=1	h=2	h=3
LightGBM (Report 3, config. A)	0.747	0.420	0.175	–0.45	–0.76	–0.89
Spark GBT	0.699	0.429	0.182	–0.50	–0.74	–0.87
Spark RF	0.687	0.454	0.214	–0.52	–0.75	–0.87

LightGBM remains best one day ahead (Table IV): GBT is –0.047 and RF –0.060 NSE behind. At h = 2 and 3 RF is ahead by +0.034 and +0.039, but on one split without a test of significance, and LightGBM used fixed hyperparameters while the MLlib models were tuned on validation; this is not evidence that the Spark models are better. All three cut flood peaks by a similar amount, so changing the engine does not remove the limitation of daily predictors. GBT chose the shallowest and slowest grid point at every horizon (depth 3, step 0.05, 150 iterations), which suggests extending the grid towards simpler models; RF chose the deepest trees (12 levels, 200 trees).

Engineering note. In the first run both executors (1,500 MB) died with exit code 52 while fitting RF, most likely because MLlib reserves 256 MB of node statistics per task by default and four tasks per executor exceeded its execution memory. Setting `maxMemoryInMB` to 64 and the executor memory to 1,800 MB fixed it. The grid search and refits took 1,686 s on 8 cores in total (354.4–364.9 s per horizon for GBT, 188.8–225.1 s for RF); a LightGBM fit of all three horizons takes a median of 10.2 s on two threads (Section IV-E).

B. *Full data of the main model, walk-forward*

Section III-A uses the smaller Report 2 panel. `spark_train_full.py` retrains both MLlib models on exactly the data of the main LightGBM model E: the extended panel from 1998, its 88 predictors, the same rows and target. Hyperparameters are chosen on validation as before (training 1998–2015, validation to June 2022, 36 fits), the refit is scored once on the test period, and the chosen settings then run through the 13 yearly folds of Table VIII, every fit as an MLlib job on the cluster.

TABLE V
MLLIB ON THE FULL DATA OF THE MAIN MODEL: NSE ON THE TEST PERIOD AND MEAN NSE OVER 13 WALK-FORWARD FOLDS 2013–2025

Model	Test h=1	h=2	h=3	WF h=1	h=2	h=3
LightGBM E (main model)	0.747	0.499	0.236	0.761	0.437	0.206
Spark GBT	0.692	0.503	0.215	0.717	0.382	0.116
Spark RF	0.717	0.506	0.224	0.712	0.390	0.179

On the test period MLlib is level with E at h = 2 (GBT +0.004, RF +0.007) and behind at h = 1 and 3 (Table V). Across the folds both models are worse at every horizon, with the 95% bootstrap interval of the difference below zero and Wilcoxon $p < 0.05$: one day ahead GBT –0.044 [–0.068; –0.022] (wins/losses 1/12) and RF –0.049 [–0.070; –0.030] (0/13); two days ahead –0.055 and –0.047; three days ahead –0.090 and –0.027. The small test gain at h = 2 is therefore the noise of one split. Peak bias ranges from –0.30 at h = 1 to –0.84 at h = 3 and neither model catches a level-1 day three days ahead, so more years and a distributed learner do not remove the peak underestimation. A likely reason for the gap is the learner, not the data: MLlib grows trees level by level to a fixed depth, LightGBM leaf by leaf with regularisation. Both steps took 116.7 minutes on 8 cores.

IV. DEEP LEARNING

A. Models

The LSTM [9] reads a window of L = 14 or 30 days of nine raw daily variables (discharge, rainfall per band and basin mean, antecedent precipitation index, season code, flood-season flag) instead of hand-made lag columns, and predicts log discharge at t + 1, t + 2 and t + 3 jointly, as in LSTM rainfall–runoff models [11]. Options are to predict the change from today's discharge and to use a peak-weighted loss (MSE weighted by discharge over its training mean, clipped). An MLP on the LightGBM lag columns is the deep-learning baseline. All models keep the LightGBM protocol: inputs standardised with training statistics only, early stopping on validation, then a refit on training plus validation for the selected number of epochs. The reference LSTM (L = 30, MSE) has 19,395 parameters and 2,162 training windows; 8 configurations were run with three seeds each.

B. Distributed training through Spark

The training function `train(config)` is pure: it reads only its configuration and the panel path. When the world size exceeds one it starts a gloo process group, wraps the model in DDP and shards samples with a distributed sampler, so gradients are averaged across ranks after every batch [6]. It was run in four ways: a single process; TorchDistributor in local mode (torchrun on the driver); TorchDistributor on the cluster, which starts a barrier stage with one task per executor; and a barrier stage written for this project, in which each task obtains the address of task 0 from the barrier context, sets the rank and world size and calls `train()` directly. The last path exists because TorchDistributor fails on Windows: torchrun builds its rendezvous store with libuv, which the Windows build of PyTorch lacks, and disabling it through the documented environment variable does not help. Table VI lists the runs.

TABLE VI
DISTRIBUTED LSTM TRAINING (L = 30, MSE, SEED 42). TRAIN() = RANK 0 FROM DATA LOADING TO EVALUATION; WALL = CALL TO RESULT, SPARK SESSION START EXCLUDED; RANGES OVER REPEATED RUNS

Run (where)	Proc. × threads	Global batch	Best epoch	train() (s)	Wall (s)
Single process (host)	1 × 4	64	10	16.1–18.2	16.1–18.2
Barrier, local[2] (host)	2 × 2	128	13	17.6–17.7	26.3–26.8
Barrier, local[2] (host)	2 × 2	64	8	17.8	26.6
TorchDistributor, local (host)	2 × 2	—	—	failed: no libuv	—
Single process (container)	1 × 4	64	10	9.0–9.5	9.0–9.5
TorchDistributor, local mode (container)	2 × 2	128	13	9.9–10.1	16.3–16.4
TorchDistributor, 2 workers (cluster)	2 × 4	128	13	9.9–10.1	22.6–23.2
TorchDistributor, 2 workers (cluster)	2 × 4	64	8	10.3	23.5
Barrier, 2 workers (cluster)	2 × 4	128	13	9.9–10.4	15.0–15.5

Four results follow. (1) The single-process run reproduces seed 42 of the deep-learning experiment exactly (NSE 0.709, 0.432, 0.180) on the host and in the container. (2) With the same number of ranks and global batch, the result does not depend on how training is distributed: all two-rank runs with batch 128 give 0.712, 0.413, 0.160 (with batch 64: 0.704, 0.413, 0.165); barrier mode and TorchDistributor are bit-identical on the cluster, and host and container agree to four decimals. (3) DDP changes the learning curve, not the skill: +0.003, –0.019 and –0.020 NSE against the single process, within about one seed standard deviation (0.016, 0.018, 0.021). (4) Distribution is slower: on the cluster `train()` takes 9.9–10.1 s against 9.0–9.5 s for one process, and TorchDistributor adds 12.7–13.1 s to start the barrier stage and torchrun (5.1 s for the lighter barrier path). With a small model and a few dozen batches per epoch, synchronising gradients after every batch costs more than the split saves.

C. Deep learning against LightGBM

On the single split some of the 8 configurations beat LightGBM at h = 2 and 3 (Fig. 3; best: a peak-weighted MLP, 0.490 against 0.420 at h = 2), but the LSTM selected on validation lost at all three horizons (0.709, 0.404, 0.171 against 0.747, 0.420, 0.175); the better configurations were identified only after seeing the test set. Under the 13-fold yearly walk-forward of Report 3 (Table VII, Fig. 4) no deep model beats LightGBM consistently. The LSTM with MSE loss is level at h = 1 (+0.021, interval includes zero) and slightly worse at h = 2–3; the peak-weighted LSTM loses clearly at h = 3 (–0.229, p = 0.033). The single-split advantage at h = 2–3 appears only in 2022–2025, the years that split happened to test; in 2013–2021 every deep model loses on average at h = 2 and 3.

TABLE VII
DEEP LEARNING AGAINST LIGHTGBM, 13 YEARLY FOLDS 2013–2025: ΔNSE MEAN, 95% BOOTSTRAP INTERVAL, FOLDS WON/LOST; POOLED CSI

Model	h = 1	h = 2	h = 3	CSI level 1, h = 1
LightGBM (NSE)	0.743	0.334	0.118	0.44
LSTM L=30, MSE	+0.021 [–0.014; +0.058] 8/5	–0.025 [–0.117; +0.032] 7/6	–0.026 [–0.101; +0.021] 7/6	0.49
LSTM L=30, peak-weighted	–0.007 [–0.065; +0.037] 8/5	–0.107 [–0.239; –0.008] 5/8	–0.229 [–0.411; –0.073] 5/8	0.49
MLP on lags, peak-weighted	+0.000 [–0.031; +0.037] 5/8	–0.065 [–0.142; +0.002] 5/8	–0.118 [–0.255; –0.015] 6/7	0.49

Deep learning has two genuine strengths one day ahead: it detects more level-1 flood days (POD 0.59–0.63 against 0.51; pooled CSI 0.49 against 0.44) and cuts peaks less (peak bias –0.21 to –0.26 against –0.38). It is also far less stable: the seed-to-seed spread is 0.03–0.20 NSE against at most 0.01 for

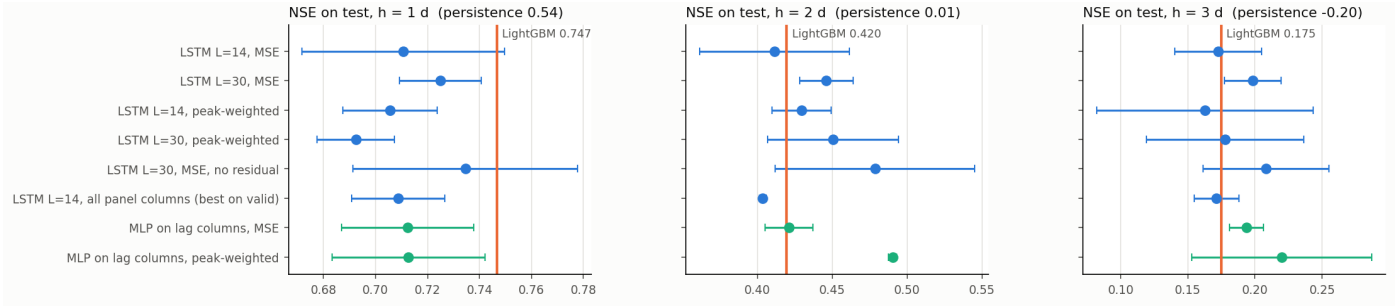


Fig. 3. Single split (test from July 2022, refit on train plus validation): NSE of the 8 deep-learning configurations, mean and standard deviation over three seeds; the vertical line is LightGBM. Only the LSTM on all panel columns was selected on validation; the others are shown for completeness.

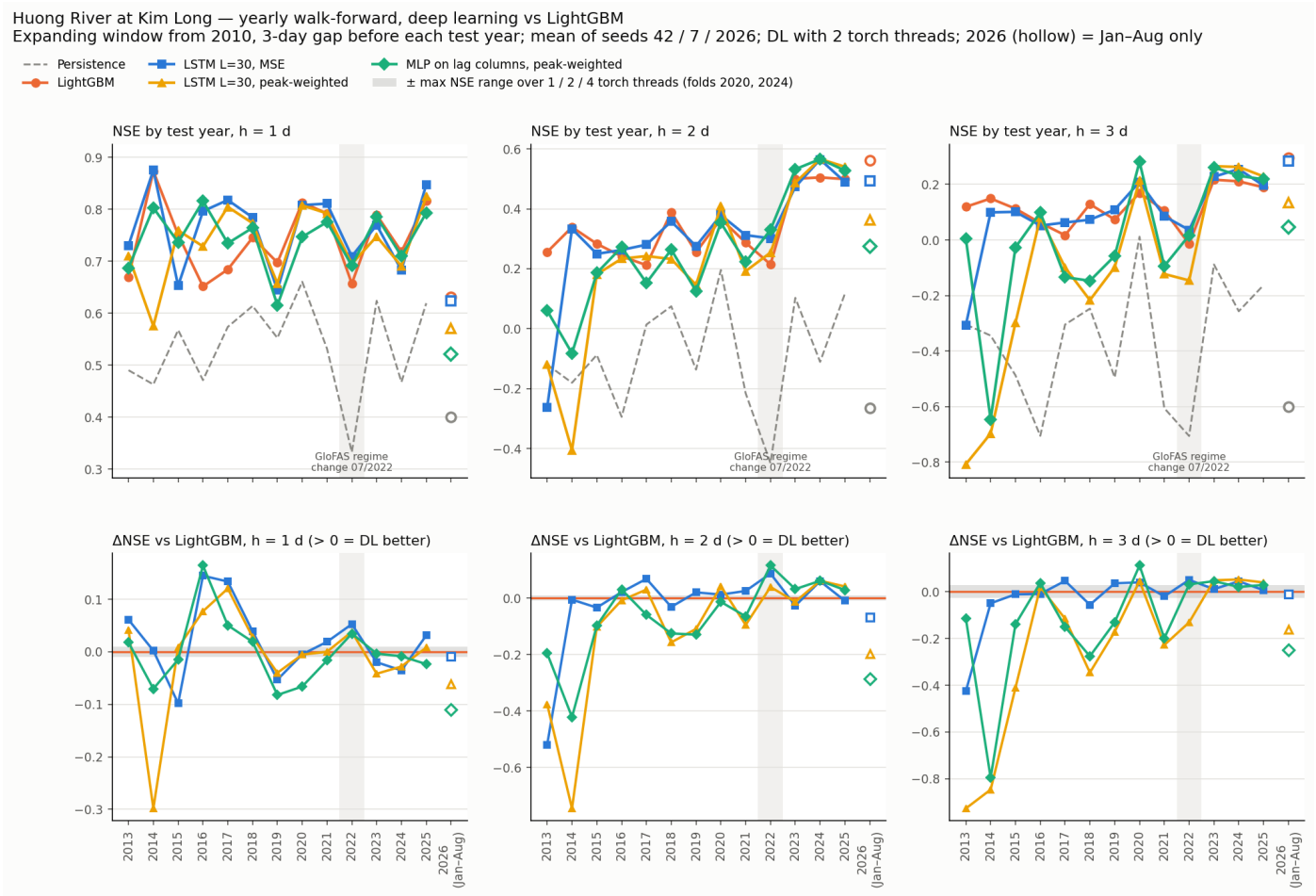


Fig. 4. Deep learning against LightGBM under the yearly walk-forward (panel from 2010, 3-day gap). Top: NSE per test year; bottom: ΔNSE, positive when the deep model is better. Shaded: GloFAS regime change (07/2022); hollow markers: January–August 2026, excluded from the summary. The grey band is the largest change caused by the torch thread count.

LightGBM, and the torch thread count moves the MLP by up to 0.028 while leaving the LSTM unchanged. LightGBM remains the main model; the deep models are candidates for a detection-oriented ensemble.

D. Deep learning as an ensemble member

Since the LSTM detects more flood days one day ahead, it was tested as a member of an ensemble with the main model of Report 3 (LightGBM configuration E, training from 1998) on 13 yearly folds, together with two non-deep remedies for peak cutting; acceptance criteria were fixed before running (a gain at h = 2 or 3 with p below 0.05, no loss at h = 1). A plain 50/50 average of E and the LSTM (L = 30, MSE, three seeds, weight fixed in advance) gives the only significant gain of the series, at h = 1: +0.029 NSE [+0.006; +0.056], 10/3 folds, Wilcoxon p = 0.048, with fewer false level-1 alarms (FAR 0.25 against 0.29). It loses at h = 2 (–0.022) and h = 3 (–0.023, p = 0.021) (Table VIII). Peak-weighted sample weights are not significant and not monotone in their strength, and an isotonic correction of the forecasts fitted on earlier years is harmful (–0.027 at h =

3, 1/12 folds). None meets the criteria, so E remains the main model; a horizon-specific blend (LSTM only at h = 1) was chosen after seeing these results and must be confirmed on new data. The value of deep learning here is as a complementary member one day ahead, not as a replacement.

TABLE VIII
CONFIGURATION E AND TWO VARIANTS, 13 YEARLY FOLDS: ΔNSE (FOLDS WON/LOST, WILCOXON P)

Variant	h = 1	h = 2	h = 3
E alone (NSE)	0.761	0.437	0.206
E + LSTM, 50/50	+0.029 (10/3, p 0.048)	–0.022 (2/11, p 0.068)	–0.023 (2/11, p 0.021)
E, peak weights $\alpha = 3$	+0.011 (9/4, p 0.13)	+0.012 (9/4, p 0.22)	+0.001 (6/7, p 0.89)

E. Where parallelism pays: independent fits on Spark

The deep-learning walk-forward needed 168 independent fits (four models, fourteen test years including the partial year 2026, three seeds), 4,049 s in sequence; a median LSTM fit takes 32.8–34.5 s, against 10.2 s for LightGBM. A local pool of 6 processes finished them in about 13 minutes. This task parallelism maps directly onto Spark and was implemented for the LightGBM runs of Section IV-D. Each of the 141 runs (configuration × test year × seed, three horizons each, about 14.8 s) becomes one Spark task that calls the same function as the local run; the code is in `spark_walkforward.py` and `wf_peak.py`. On the two-worker cluster with 8 cores the runs took 270.7 s against 2,081 s of summed task time, a speed-up of 7.7 (96% of the ideal 260.2 s). The forecasts match the local runs on all 154,494 values to 2.3×10^{-13} m^3/s: equal up to rounding, not bit-identical, since LightGBM is built differently for Linux and Windows.

TABLE IX
TASK-PARALLEL WALK-FORWARD OF LIGHTGBM (PEAK EXPERIMENTS)

Way of running the 141 runs	Cores	Wall (s)
In sequence (sum of task times)	1	2,081
Process pool on the Windows host	8	329.9
Spark, 2 workers × 4 cores	8	270.7

Spark finished 1.2 times faster than a pool with the same number of processes on the host. This should not be read as Spark beating a process pool: the tasks ran under Linux in the containers and the pool under Windows, so part of the gap is likely the operating system and the LightGBM build rather than Spark. What the run shows is that independent fits scale almost linearly on the cluster and give the same forecasts; on several real machines this is the part of the project that would keep scaling.

V. SCALING AND HONEST ASSESSMENT

A. Measured cost

At this data size Spark is slower than Polars (Fig. 2). For the ETL, the two-worker cluster takes 11.6 times as long as Polars on the host, and Spark on the host through Arrow 17.0 times; inside the same container, with the same mounted disk, the best Spark run is 3.5 times slower than Polars. For the hourly features the ratio is 7.3 on the cluster and 32.8 on the host (Table II). Parallelism does work inside Spark: one to four cores speeds the ETL up 2.6 times on the cluster and 2.8 times in local mode. Going from four cores on one worker to eight on two does not (38.2 s against 40.5–41.2 s), and the full satellite record, 15 times larger, shows the same pattern (Table III). Distributed LSTM training is slower than one process (Section IV-B). The exception is the task-parallel walk-forward (Section IV-E): 7.7 times faster than in sequence on 8 cores, with the same forecasts.

B. Why

- **I/O through the mount.** Reading the 768 files from the Windows disk into Docker alone takes 56–63 s with one core and 14–21 s with eight, 35–65% of the ETL time; Polars on the host reads the same files natively and is 2.7 times faster than Polars in the container.
- **Skewed partitions.** Only 64 cell keys are hashed into 16 partitions; the largest holds 920,376 rows, 2.1 times the mean of 436,326, and the slowest task sets the time of the whole stage.
- **One physical machine.** Both workers share the host's cores, memory bus and disk, so adding a worker adds scheduling and shuffling, not capacity.
- **Fixed costs.** Starting a Spark application takes 4.1–6.5 s in the containers and 18.4 s on the host, before any data are read; task scheduling and shuffles add more. For two months of PERSIANN-CCS Polars needs 0.2 s and Spark 14.0–35.7 s.
- **Small model.** With 19,395 parameters and 2,162 training windows, per-batch gradient synchronisation dominates DDP.

This is the COST argument [4]: a distributed system should first be compared with a competent single-threaded implementation, and for data that fit in memory the single machine usually wins. Run times were measured while other jobs shared the machine and vary by a few percent; this does not change the ratios.

C. When Spark would pay off

Spark becomes the right tool when the data no longer fit in one machine's memory or already live in distributed storage. Plausible extensions of this project reach that point: forecasting many river cells across Vietnam instead of one; PERSIANN-CCS at 0.04° for its full record, about 130 million rows for this basin alone (an estimate, which one machine with Polars would still handle); and an hourly-input LSTM reading 168 hourly steps of 64 cells per sample, in the spirit of multi-timescale LSTMs [12]. Task parallelism already pays at the present size (Section IV-E); it is the first job to move to

several real machines, together with hyperparameter grids, which have the same independent structure.

D. Path to the cloud (proposal, not executed)

The code is already portable: the Spark session reads the master address from an environment variable, the training function takes the panel path as a parameter, and TorchDistributor runs unmodified on Linux. A managed workspace such as Databricks Free Edition [13] could host the same jobs with Parquet replaced by Delta tables [8]. This step has not been run, and no result in this report comes from it; the free tier's limits on compute size and on installing PyTorch must be checked first.

VI. CONCLUSION

The project's pipeline now runs on a Spark cluster and gives the same answers as the single-machine version: the ETL row for row and the hourly features bit for bit, including the full satellite record of 101.0 million rows. MLlib models trained on the cluster with the full data of the main model are worse than LightGBM at every horizon over 13 walk-forward folds (one day ahead –0.044 and –0.049 NSE, $p < 0.05$). LSTM training distributed through TorchDistributor or a barrier stage reproduces the same result on one machine and on two workers, and across 13 yearly folds no deep model beats LightGBM consistently. Deep learning helps only as an ensemble member one day ahead (blend with the main model: +0.029 NSE at $h = 1$, $p = 0.048$, but losses at $h = 2$ and 3).

Recommendations follow from the measurements. (1) Keep Polars for the ETL and the hourly features: at the present size Spark is slower, also on the full record (1.8–6.6 times), and the tested Spark jobs remain the path for larger inputs. (2) Run walk-forward fits as Spark tasks: this is faster (7.7 times on 8 cores) and verified against the local forecasts, and it is what would gain from more machines. (3) Keep LightGBM configuration E as the main model and DDP out of the daily job; a horizon-specific LightGBM–LSTM blend at $h = 1$ is the next deep-learning step, to be confirmed on new data.

REFERENCES

[1] S. Harrigan *et al.*, “Daily ensemble river discharge reforecasts and real-time forecasts from the operational Global Flood Awareness System,” *Hydrol. Earth Syst. Sci.*, vol. 27, no. 1, pp. 1–19, 2023.

[2] H. Hersbach *et al.*, “The ERA5 global reanalysis,” *Q. J. R. Meteorol. Soc.*, vol. 146, no. 730, pp. 1999–2049, 2020.

[3] M. Zaharia *et al.*, “Apache Spark: A unified engine for big data processing,” *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] F. McSherry, M. Isard and D. G. Murray, “Scalability! But at what COST?,” in *Proc. 15th Workshop Hot Topics Oper. Syst. (HotOS XV)*, 2015.

[5] X. Meng *et al.*, “MLlib: Machine learning in Apache Spark,” *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

[6] S. Li *et al.*, “PyTorch distributed: Experiences on accelerating data parallel training,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3005–3018, 2020.

[7] Apache Software Foundation, “pyspark.ml.torch.distributor.TorchDistributor,” PySpark API reference. [Online]. Available: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.torch.distributor.TorchDistributor.html (accessed Nov. 2, 2026).

[8] M. Armbrust *et al.*, “Delta Lake: High-performance ACID table storage over cloud object stores,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3411–3424, 2020.

[9] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 3146–3154.

[11] F. Kratzert, D. Klotz, C. Brenner, K. Schulz and M. Herrnegger, “Rainfall–runoff modelling using Long Short-Term Memory (LSTM) networks,” *Hydrol. Earth Syst. Sci.*, vol. 22, no. 11, pp. 6005–6022, 2018.

[12] M. Gauch, F. Kratzert, D. Klotz, G. Nearing, J. Lin and S. Hochreiter, “Rainfall–runoff prediction at multiple timescales with a single Long Short-Term Memory network,” *Hydrol. Earth Syst. Sci.*, vol. 25, no. 4, pp. 2045–2062, 2021.

[13] Databricks, “Sign up for Databricks Free Edition,” Databricks on AWS documentation. [Online]. Available: https://docs.databricks.com/aws/en/getting-started/free-edition (accessed Nov. 2, 2026).